Funciones en Python

- OPERADOR WALRUS PARA ASIGNAR
- FUNCIONES
    + EJEMPLO DE MODULARIZACIÓN DE CÓDIGO
    + PARÁMETROS Y ARGUMENTOS
    + USO DE *args* Y *kwargs*
    + ANOTACIONES Y TIPADO EN FUNCIONES
    + FUNCIONES RECURSIVAS
    + FUNCIONES ANÓNIMAS: EXPRESIONES LAMBDA
    + FUNCIONES DE ORDEN SUPERIOR
    + FUNCIONES DENTRO DE FUNCIONES
    + ESPACIO DE NOMBRES Y CONTEXTOS
    + MEMOIZACIÓN
    + DECORADORES
    + DOCUMENTACIÓN DE FUNCIONES
    + FUNCIONES GENERADORAS
    + CORRUTINAS (NO)
    + FUNCIONES ASÍNCRONAS (NO)

OPERADOR WALRUS PARA ASIGNAR

- Desde la versión 3.8 tenemos el operador := (walrus se parece a los colmillos de una morsa)
- Uso: asignar variables mientras se usan en expresiones. Simplifica y mejora la legibilidad de las mismas.

```
# sin walrus

bruto = 2351
gastos = 456
neto = bruto - gastos
if neto > 0:
    print(f'Neto positivo {neto}')
else:
    print(f'Neto negativo {neto}')
```

```
# con walrus

bruto = 2351
gastos = 456
if (neto := bruto - gastos) > 0:
    print(f'Neto positivo {neto}')
else:
    print(f'Neto negativo {neto}')
```

Realiza la actividad walrus.py

FUNCIONES EN PYTHON

Función: subprograma o subrutina que forma parte de un algoritmo principal. En algunos lenguajes de programación se distingue entre funciones (devuelven valores) y procedimientos (no devuelven valor).

En Python todas las funciones devuelven None por defecto.

Objetivo de las funciones:

- Mejorar la organización del código.
- Mejorar la legibilidad del código.
- Reutilizar la lógica común en diferentes partes del código (refactorizar).

Además, ayudan a desacoplar el código (módulos independientes), facilitan el testeo, ... nos permite separar grupos de programadores según secciones lógicas de los algoritmos a implementar.

```
def foo():
    pass

def menor_valor(iter):
    return min(iter)

def iniciales(cadena, sep = ' '):
    "Devuelve las iniciales de cada palabra en la cadena separadas por sep"
    return ''.join(x[0].upper() for x in cadena.split(sep))

print('foo(): ', foo())
print('menor_valor([1, 2, 3, 4]): ', menor_valor([1, 2, 3, 4]))
print("iniciales('Natxo Febrer Senar'): ", iniciales('Natxo Febrer Senar'))
```

objeto compatible con min()

objeto compatible con método split()

valor por defecto

Realiza la actividad funciones_sencillas.py

Ejemplo de modularización de código

```
# Partiendo de una cadena de caracteres, se pretende separar la cadena mediante un separador
# y aplicar una función basada en el algoritmo de cifrado md5 a cada elemento para obtener
# su forma hexadecimal. Una vez obtenida su forma, se ha de guardar en un fichero de texto plano.
import hashlib

def extraer_identificadores(cadena_original, sep=' '):
    return cadena_original.split(sep)

def aplicar_hash(lst):
    md5s = []
    for cad in lst:
        m = hashlib.md5(cad.encode('utf-8'))
        md5s.append(m.hexdigest())
    return md5s

def escribir_a_fichero(data, nombre_fichero='out.txt'):
    with open(nombre_fichero, 'w') as f:
        for d in data:
            f.write(d + '\n')
    return nombre_fichero

cadena_original = 'texto<>a<>guardar'
identificadores = extraer_identificadores(cadena_original, sep='<>')
id_modificados = aplicar_hash(identificadores)
nombre_fichero = escribir_a_fichero(id_modificados)
```

La lógica de la ejecución está encapsulada en las funciones y no en el nivel principal del módulo.

Si cambian las especificaciones: aplicar las función de cifrado sha256 (en vez de md5) y guardar en formato JSON (en vez de texto plano).
Solo tenemos que crear las nuevas funciones que sustituyan las que ya no se quieren.

out.txt

```
1  62059a74e9330e9dc2f537f712b8797c
2  0cc175b9c0f1b6a831c399e269772661
3  fef4b7a42597267132d9f80c75e3da0d
4
```

Realiza la actividad ej_modularización.py

Ejemplo de modularización de código

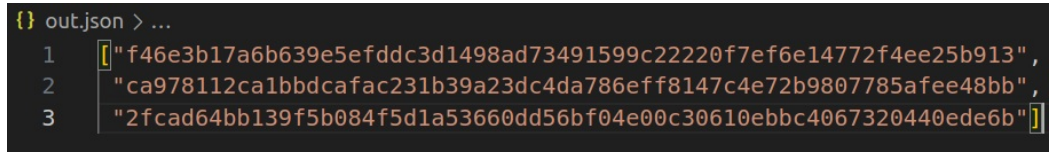
```python
import hashlib
import json

def extraer_identificadores(cadena_original, sep=' '):
    return cadena_original.split(sep)

def aplicar_hash(lst):
    hashes = []
    for cad in lst:
        m = hashlib.sha256()
        m.update(cad.encode('utf-8'))
        hashes.append(m.hexdigest())
    return hashes

def escribir_a_json(data, nombre_fichero='out.json'):
    with open(nombre_fichero, 'w') as f:
        json.dump(data, f)
    return nombre_fichero

cadena_original = 'texto<>a<>guardar'
identificadores = extraer_identificadores(cadena_original, sep='<>')
id_modificados = aplicar_hash(identificadores)
nombre_fichero = escribir_a_json(id_modificados)
```



Hemos cambiado la lógica de las funciones sin que esto afecte al programa en general.

Realiza la actividad ej_modularización2.py

Parámetros y argumentos en funciones

Parámetros de la función: nombres de las variables utilizados en la definición.

Argumentos de la función: variables que portan los objetos en la llamada.

Los parámetros son una sucesión de identificadores separados por comas. Opcionalmente pueden tener un valor por defecto usando "=" y un tipo usando ":".

Se pueden pasar argumentos posicionalmente en el orden tal y como están definidos o no posicionalmente usando el nombre de los parámetros con "=". El caracter "/" indica que los parámetros desde el comienzo hasta "/" solo se pueden usar posicionalmente.

```
def division(num, deno=1):
    return num / deno

print(division(45, 3)) # 15.0
print(division(45)) # 45.0
print(division(deno=3, num=45)) # argumentos pasados no posicionalmente
                    # usando los nombres de los parámetros 15.0
```

```
def permu( num: int, veces:float) -> float:
    return num * veces

print(permu(7, 2.0)) # 14.0
print(permu("7", 2.0)) # TypeError

def suma_pos(a, b, /):
    return a + b

print(suma_pos(1, 2)) # 3
print(suma_pos(a=1, b=2)) # TypeError
```

Realiza la actividad anotaciones_tipado.py

Parámetros y argumentos en funciones

En Python todos los argumentos son pasados por referencia. Esto significa que solo se copian los punteros (dirección de memoria) de las variables y no el contenido en sí.

Ventaja: mover objetos grandes es muy barato en términos de memoria.

Desventaja: si el objeto es mutable y su contenido se modifica dentro de la función, se modificará en las referencias que haya de ese objeto fuera de la función.

```
a = 'Hola'; b = ' Mundo'
def suma(elem1, elem2):
    elem1 += elem2
    return elem1

c = suma(a, b) # al ser elementos no mutables a no se modifica
print(c) # Hola Mundo
print(a) # Hola

a = [1, 2, 3]; b = [4, 5]
c = suma(a, b) # al ser elementos mutables a se modifica
print(c, id(c)) # [1, 2, 3, 4, 5] misma referencia que a
print(a, id(a)) # [1, 2, 3, 4, 5] misma referencia que c
```

Realiza la actividad argumentos_referencia.py

Parámetros y argumentos en funciones

Cuando se usan valores por defecto en las funciones: hay que tener la precaución de que éstos no sean de tipo mutable. Se recomienda cambiarlos por None y comprobar que el valor como argumento es None para inicializar.

Realiza la actividad argumentos_referencia.py

```
def añadir_valor(elem, lst=[]):
    lst.append(elem)
    return lst

print(añadir_valor(1)) # [1]
print(añadir_valor(2)) # [1, 2]
print(añadir_valor(3)) # [1, 2, 3]

def añadir_valor_seguro(elem, lst=None):
    if lst is None:
        lst = []
    lst.append(elem)
    return lst

print(añadir_valor_seguro(1)) # [1]
print(añadir_valor_seguro(2)) # [2]
print(añadir_valor_seguro(3)) # [3]
```

Uso de args y kwargs

Los argumentos de las funciones se pueden pasar posicionalmente o usando clave-valor (parámetro=valor).
En la definición de las funciones se puede especificar cada grupo y cómo trabajar con ellos:

- argumentos posicionales: se descomponen usando un '*' (operador que descompone una secuencia) y se les denomina args.
- argumentos clave-valor: se descomponen usando '**' (operador que descompone un diccionario) y se les denomina kwargs.

Ejemplos:

```
def mi_funcion(*args, **kwargs):
    print(f'args: {args}, kwargs: {kwargs}')

mi_funcion() # args: (), kwargs: {}
mi_funcion(42) # args: (42,), kwargs: {}
mi_funcion(vehículo='Coche') # args: (), kwargs: {'vehículo': 'Coche'}
mi_funcion(345, 'Juan', modo='Vuelo', flag='Activado')
# args: (345, 'Juan'), kwargs: {'modo': 'Vuelo', 'flag': 'Activado'}
```

Realiza la actividad args_kwargs.py

En Python, los parámetros de las funciones son estrictos, y si no se define la extensión de los parámetros con args y kwargs, las funciones elevarán excepciones al intentar ser ejecutadas con más parámetros de los definidos.

Uso de args y kwargs

args y kwargs son útiles a la hora de extender clases o de ejecutar funciones que hagan uso de algunos parámetros que se provean solo en la llamada de la función principal.

```
# función general que suministra los parámetros como argumentos
# de las funciones internas
def calculo_general(**kwargs):
    ret_s = suma(**kwargs)
    ret_d = division(**kwargs)
    return ret_s * ret_d

# define por nombre cada parámetro que se debe usar
# los parámetros que no aparecen están en **kwargs
def suma(op1, op2, **kwargs):
    return op1 + op2

def division(num, deno, **kwargs):
    return num / deno


res = calculo_general(op1=5, op2=9, num=8, deno=2)
print(res) # 56.0
```

Realiza la actividad args_kwargs.py

Uso de args y kwargs

El ejemplo anterior con *args:

```
def calculo_general2(*args):
    ret_s = suma2(*args)
    ret_d = division2(*args)
    return ret_s * ret_d

def suma2(op1, op2, *args):
    return op1 + op2

# al acceder posicionalmente, se añaden los dos primeros parámetros
# aunque no se utilicen
def division2(a1, a2, num, deno):
    return num /deno

res = calculo_general2(5, 9, 8, 2)
print(res) # 56.0
```

Realiza la actividad args_kwargs.py

Por lo general, cuando se usa *args o *kwargs se pierde legibilidad del código y se permite que cualquier parámetro sea utilizado en todas las funciones. El código se vuelve poco mantenible. Este concepto nos servirá cuando ahondemos en las CLASES y HERENCIA.

Se recomienda nombrar explícitamente los parámetros a usar en cada función.

ANOTACIONES Y TIPADO EN LAS FUNCIONES

Pyhton es un lenguaje dinámicamente tipado: la comprobación de los tipos se hace en tiempo de ejecución.

A partir de Python 3.5 se puede anotar los tipos de los parámetros y valores devueltos de las funciones.

Las anotaciones de deben tomar como sugerencias (hints) o intención del desarrollador ya que Python no modifica el comportamiento de la ejecución del código. Hay editores avanzados que leen las anotaciones y comprueban estáticamente los tipos.

```
def func(x: int, y:str, z:dict)->bool:
    pass

def foo(elem: float=43.7)->float:
    pass

print(func.__annotations__)

print(foo.__annotations__)
```

Las anotaciones nos permiten ver el uso que pretende el desarrollador para dicha función.

El método mágico __annotations__ permite ver las anotaciones de una función.

Realiza la actividad anotaciones_tipado.py

ANOTACIONES Y TIPADO EN LAS FUNCIONES

Los tipos de datos disponibles por defecto son los de la librería estándar (int, float, str, etc.) y las clases definidas en el código y librerías externas.

Si queremos poder anotar estructuras más complejas o tipos colección (listas, diccionarios, ...) podemos usar la librería typing que define gran cantidad de tipos (Any, List, Dict, Container, Collection, Counter, Generator, ...).

A partir de Python 3.9 los tipos colección (listas, tuplas, diccionarios, ...) se pueden usar como anotaciones sin necesidad de la librería typing.

```
from typing import Counter, Dict, List, Tuple

def bar2(elem1: Counter[str], elem2: Dict[int, float])->List[Tuple[str]]:
    pass
```

Realiza la actividad anotaciones_tipado.py

FUNCIONES RECURSIVAS

Son funciones que se llaman a sí mismas para completar su lógica.

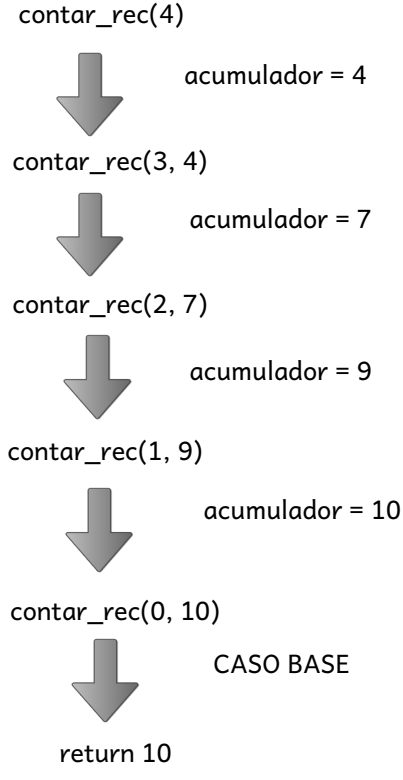
Estructuralmente constan del siguiente patrón:

- Caso base: por el que termina o comienza la ejecución.
- Llamada recurrente (recursiva): la función se llama a sí misma.

```
def contar_rec(elem, acumulador=0):
    if elem == 0:
        return acumulador # caso base
    else:
        acumulador += elem
        return contar_rec(elem - 1, acumulador) # llamada recursiva

print(contar_rec(4)) # 10
print(contar_rec(40)) # 820
```

Realiza la actividad recursivas.py



FUNCIONES RECURSIVAS. EJEMPLOS.

```
# EJEMPLO1: función que acepte 2 parámetros
# como argumentos, pero en caso de no estar
# inicializados, se definen internamente y se
# llama de nuevo a la función con los nuevos valores.
def simple_func(elem1, elem2=None):
    if elem2 is None:
        return simple_func(elem1, elem2=3) # llamada recursiva
    return elem1 * elem2

print(simple_func(3)) # 9
print(simple_func(3, 4)) # 12

# EJEMPLO2: función que simplifica una estructura compuesta de
# listas anidadas de N niveles. La función debe devolver una
# versión aplanada de la original, es decir, una lista simple.
def aplanador(llst):
    lista_plana = []
    for elem in llst:
        if isinstance(elem, list):
            lista_plana.extend(aplanador(elem))
        else:
            lista_plana.append(elem)
    return lista_plana

print(aplanador([1, 2, 3, [4, 5, 6], [7, [8, [9]]]]))
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
import sys

# EJEMPLO3: función que devuelve el valor de la posición n
# de la secuencia de Fibonacci. Cada elemento se calcula con
# la suma de los dos elementos anteriores; el primer y segundo
# elemento son los valores 0 y 1 respectivamente.
def fibo(n):
    if n <= 1:
        return n # caso base
    else:
        return fibo(n -1) + fibo(n-2) # llamadas recursivas

print([fibo(x) for x in range(26)])
# [0, 1, 1, 2, 3, 5, 8, 13, 21, ... , 28657, 46368, 75025]

print('límite recursión: ', sys.getrecursionlimit())
# límite recursión:  1000
```

Excepción RecursionError: cuando el número de llamadas recursivas excede un límite máximo (sys.getrecursionlimit()). Otro problema, es tener el algoritmo mal diseñado y provocar recursiones infinitas.

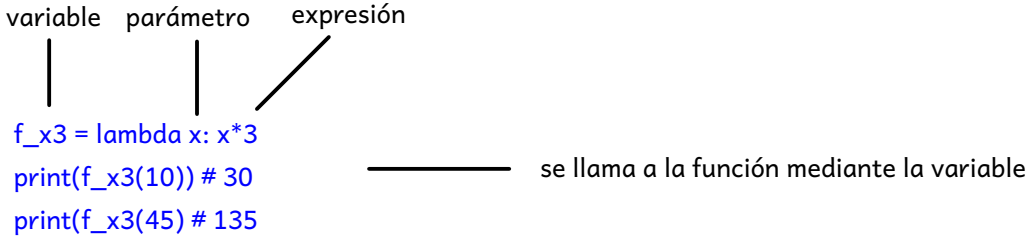
FUNCIONES ANÓNIMAS: EXPRESIONES LAMBDA

Expresiones lambda: permiten crear funciones anónimas o sin nombre. No se permite que tengan sentencias de código o anotaciones. Solo se permite que tengan expresiones.

Uso:

- Argumentos para otras funciones. Solo se pueden llamar una vez en la ejecución.
- Almacenamiento en variables. Se pueden llamar más de una vez al reusar la variable.

Las funciones anónimas se utilizan en las funciones de ordenación (sort, sorted), min, max, manipulación (map) o filtrado (filter). Ejemplos:



sorted(iterable, /, *, key=None, reverse=False)

función predefinida: retorna una nueva lista que contiene todos los items del iterable en orden ascendente.

Se puede añadir una función de ordenación (Key function) para personalizar la ordenación.

El flag reverse establece el orden inverso.

```
lst = ['a1', 'a12', 'a100', 'a5']
print(sorted(lst)) #ordenado por caracter: ['a1', 'a100', 'a12', 'a5']

print(sorted(lst, key=lambda x: int(x[1:]))) #ordenación numérica correcta
# ['a1', 'a5', 'a12', 'a100']
```

FUNCIONES ANÓNIMAS: EXPRESIONES LAMBDA

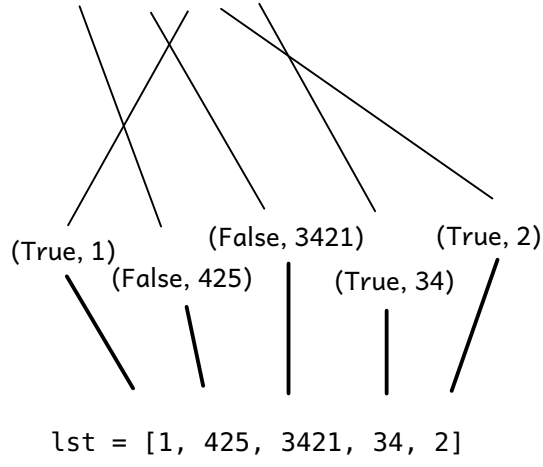
sort(self, /, *, key=None, reverse=False)

Método que se aplica a un objeto list y lo ordena ascendentemente. Retorna None.

Se puede usar una key function.

El flag reverse permite ordenar de manera descendente.

```
lst = [1, 425, 3421, 34, 2]
lst.sort(key=lambda x: (x < 400, x)) #ordenaciones por varios criterios usando tuplas
print(lst) # [425, 3421, 1, 2, 34]
```



(x < 400, x)

(1er criterio, 2do criterio)

NOTA: False < True

min(iterable, *[, default=obj, key=func]) -> value

min(arg1, arg2, *args, *[, key=func]) -> value

Función predefinida:

Con el argumento iterable retorna el item más pequeño. Si el iterable está vacío se retorna el objeto que indica default=

Con dos o más argumentos retorna el más pequeño.

Permite el uso de key function para decidir de manera personalizada el más pequeño.

FUNCIONES ANÓNIMAS: EXPRESIONES LAMBDA

```
print(lst) # [425, 3421, 1, 2, 34]
print(min(lst)) # 1
print(min(lst, key=lambda x: x < 50)) # 425
```

False, False, True, True, True

[425, 3421, 1, 2, 34]

max(iterable, *[, default=obj, key=func]) -> value

max(arg1, arg2, *args, *[, key=func]) -> value

Función predefinida:

Con el argumento iterable retorna el item más grande. Si el iterable está vacío se retorna el objeto que indica default=

Con dos o más argumentos retorna el más grande.

Permite el uso de key function para decidir de manera personalizada el más grande.

```
print(lst) # [425, 3421, 1, 2, 34]
print(max(lst, key=lambda x: x**2 > 500)) # 425
print(max(lst, key=lambda x: x * 2 < 50 and not x % 2)) # 2
```

True, True, False, False, True

[425, 3421, 1, 2, 34]

False, False, False, True, False

[425, 3421, 1, 2, 34]

class map(object)

map(func, *iterables) --> map object

Crea un iterador que computa la función usando como argumentos cada uno de los iterables. Para cuando el último iterable es consumido.

FUNCIONES ANÓNIMAS: EXPRESIONES LAMBDA

```
print(list(map(lambda x: x + 34, range(5))))
# [34, 35, 36, 37, 38]
print([x + 34 for x in range(5)]) # por comprensión
# [34, 35, 36, 37, 38]

print(list(map(lambda x: x.title(), ['juan', 'pedro', 'ana'])))
# ['Juan', 'Pedro', 'Ana']
print([x.title() for x in ['juan', 'pedro', 'ana']]) # por comprensión
# ['Juan', 'Pedro', 'Ana']
```

Una alternativa al uso de lambdas en las funciones map o filter es el uso de listas por comprensión las cuales permiten manipular los elementos y filtrarlos.

class filter(object)
filter(function or None, iterable) -> filter object
Retorna un iterator produciendo (yielding) aquellos items del iterable para los que function(item) es true. Si function es None, retorna los items que son true.

```
print(list(filter(lambda x: not x % 5, range(100))))
# [0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95]

print([x for x in range(100) if not x % 5]) # por comprensión
# [0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95]

print(list(filter(lambda x: 'e' in x.lower(), ['Perico', 'Papá', 'Mamá'])))
# ['Perico']
```

Realiza la actividad lambda.py

FUNCIONES ANÓNIMAS: EXPRESIONES LAMBDA

Un uso destacable de las funciones lambda es para crear funciones parciales.
Las funciones parciales tienen algunos parámetros asignados a valores y otros a la espera de ser instanciados cuando se llame a la función.

```
# función parcial que genera funciones dinámicas que multiplican
# el argumento final por el argumento que se usa al inicializar
# la función inicial
def f_principal(n):
    '''retorna una función anónima con el parámetro n fijado,
    x se rellena en la llamada a la función
    '''
    return lambda x : x * n

multi_9 = f_principal(9) # n queda instanciado a 9
print(multi_9(10)) # x toma el valor de 10
# 90
print(multi_9(80)) # x toma el valor de 80
# 720

multi_5 = f_principal(5) # n queda instanciada a 5
print(multi_5(10)) # x toma el valor de 10
# 50
print(multi_5(80)) # x toma el valor de 80
# 400
```

Para crear funciones parciales se recomienda usar la función partial del módulo functools.

Las funciones anónimas son muy útiles para casos puntuales: ordenación, filrado, ... para el resto de cosas son más rápidas las funciones normales.

Realiza la actividad funciones_parciales.py

FUNCIONES DE ORDEN SUPERIOR

Son funciones que aceptan otras funciones como parámetros. Se usan en la programación funcional.

Ejemplos:

```
import math

def op(a, b, func):
    return func(a, b)

def raiz_de_par(x):
    if not x % 2:
        return math.sqrt(x)
    return 0

def mul_de_par(x, multi=5):
    if not x % 2:
        return x * multi
    return 0

def aplicar_func(lst, func):
    # lista por comprensión
    return [func(x) for x in lst]
```

Argumentos que se le pasarán a la función

Nombre de la función

```
r1 = op(45, 75, int.__add__)
r2 = op(45, 75, int.__mul__)
print(r1, r2) # 120 3375

r3 = aplicar_func(range(20), mul_de_par)
print(r3) # [0, 0, 10, 0, 20, 0, 30, 0, 40, 0, 50, 0, 60, 0, 70, 0, 80, 0, 90, 0]

r4 = aplicar_func(range(20), raiz_de_par)
print(r4)
# [0.0, 0, 1.4142135623730951, 0, 2.0, 0, 2.449489742783178, 0,
# 2.8284271247461903, 0, 3.1622776601683795, 0, 3.4641016151377544,
# 0, 3.7416573867739413, 0, 4.0, 0, 4.242640687119285, 0]

# función parcial, x se especifica más adelante
mul_de_par_8 = lambda x: mul_de_par(x, multi=8)
r5 = aplicar_func(range(20), mul_de_par_8)
print(r5) # [0, 0, 16, 0, 32, 0, 48, 0, 64, 0, 80, 0, 96, 0, 112, 0, 128, 0, 144, 0]
```

Método __add__ de int

Método __mul__ de int

Realiza la actividad func_orden_superior.py

FUNCIONES DENTRO DE FUNCIONES

Ya vimos en funciones lambda que se permite que una función devuelva otra función.

Python también permite definir una función dentro de otra función: éstas crean un contexto propio con sus propias variables locales.

Clausura o cerramiento: contexto propio de una función definida dentro de otra función. Esto permite el encapsulamiento de funciones.

Ejemplo:

```
# definir funciones dentro de funciones: encapsulamiento de funciones
def telefono_con_prefijo(prefijo_pais):
    def f_telefono(numero_tlf):
        return f'+{prefijo_pais} {numero_tlf}'
    return f_telefono

tlf_españa = telefono_con_prefijo(36)
tlf_aleman = telefono_con_prefijo(96)
tlf_andorra = telefono_con_prefijo(376)

print(tlf_españa(649594922)) # +36 649594922
print(tlf_aleman(649594922)) # +96 649594922
print(tlf_andorra(649564622)) # +376 649564622
```

La función telefono_con_prefijo genera dinámicamente funciones con prefijos diferentes

Realiza la actividad clausuras.py

FUNCIONES DENTRO DE FUNCIONES

```
def beneficios(gastos:list, ingresos:list, tramos_impuestos):
    def obtener_impuesto(ingreso):
        impuesto_actual = tramos_impuestos[0][1]
        for hasta_valor, impuesto in tramos_impuestos:
            if hasta_valor > ingreso:
                break
            else:
                impuesto_actual = impuesto
        return impuesto_actual

    def calculo_neto(ingreso_bruto):
        impuesto = obtener_impuesto(ingreso_bruto)
        return ingreso_bruto - (ingreso_bruto * impuesto)

    gastos_totales = sum(gastos)
    ingresos_netos = sum(map(calculo_neto, ingresos))
    return ingresos_netos - gastos_totales

gastos = [22, 314, 32, 52]
ingresos = [45, 623, 12, 90]
tramos_impuestos = [ (20, 0.06), (50, 0.08), (200,0.1), (500,0.2), (float('Inf'), 0.21)]
balance_final = beneficios(gastos, ingresos, tramos_impuestos)
print(balance_final) # 214.77999999999986
```

Realiza la actividad clausuras.py

La lógica de la función principal queda descrita en solo tres líneas de código y hace uso de funciones internas.

Estas funciones internas usan los argumentos de la función principal y hacen que la lógica pueda ser leida con facilidad.

ESPACIO DE NOMBRES Y CONTEXTOS

En Python, las variables definidas a nivel de fichero de texto o de módulo son variabes globales. Las variables que se definen dentro de una función son variables locales.
Si una función intenta leer el valor de una variable global, definida anteriormente en el flujo del programa, no tendrá problema. Si intenta modificar el valor de la misma ocurre una excepción NameError (si se intenta actualizar) o UnboundLocalError (si se intenta incrementar).
Para poder modificar las variables globales desde una función es necesario usar la palabra reservada global.
Ejemplo:

```
v_global = 10
def foo(x):
    return x + v_global # podemos leerla

def foo2(x):
    v_global += 10 # actualización no permitida
    return v_global + x

def foo3(x):
    global v_global # permite leer y actualizar
    v_global += 10
    return v_global + x


print(foo(5)) # 15
print(foo2(5)) # UnboundLocalError: local variable 'v_global referenced before assignment
print(foo3(5)) # 25
```

La considera variable local

Realiza la actividad variables_globales_locales.p

ESPACIO DE NOMBRES Y CONTEXTOS

Si una función interna es definida dentro de una función, se crean dos contextos diferentes: el de la función principal (similar al contexto global) y el de la función interna (similar al contexto local). Pero, en este caso en vez de global usamos la palabra reservada nonlocal. Por ejemplo:

```
def foo_correct():
    ruedas = 4
    def pinchar_rueda():
        nonlocal ruedas # permite acceder al contexto superior
        ruedas -= 1
    print(f'Número de ruedas: {ruedas}')
    pinchar_rueda()
    print(f'Número de ruedas: {ruedas}')

foo_correct()
# Número de ruedas: 4
# Número de ruedas: 3

def pecera():
    pez = 'Dori'
    x = 5
    print(f'Locals: {locals()}')
    print(f'Globals: {globals()}')

pez = 'Nemo'
pecera()
```

Realiza la actividad variables_globales_locales.py

Con nonlocal Python busca una variable con el mismo nombre en los contextos superiores hasta llegar al contexto global.

Para inspeccionar las variables definidas en cada contexto se usan las funciones locals() y globals()

Contexto local

Locals: {'pez': 'Dori', 'x': 5}

Globals: {'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <_frozen_importlib_external.SourceFileLoader object at 0x7e5072bbd990>, '__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>, 'foo_correct': <function foo_correct at 0x7e5072c87e20>, 'pecera': <function pecera at 0x7e5072c87eb0>, 'pez': 'Nemo'}

Contexto global

MEMOIZACIÓN

Con funciones internas se pueden hacer lógicas interesantes: crear una minicaché interna para cálculos posteriores. Esta técnica se llama memoization. Consiste en guardar en memoria de la función principal los resultados de ejecuciones de la función interna.

Muy útil en situaciones en las que para llegar a un resultado se necesitan los resultados anteriores.

```
import timeit

def fibo(n):
    if n <= 1:
        return n # caso base
    else:
        return fibo(n-1) + fibo(n-2)
```

```
def memoize(f):
    '''Crea una caché del último resultado de f'''
    mem = {} # dict
    def mem_func(n):
        if n not in mem:
            mem[n] = f(n)
        return mem[n]
    return mem_func
```

```
mem_fibo = memoize(fibo) # versión memoized de fibo
```

```
nc = timeit.timeit('fibo(32)', globals = globals(), number = 1)
# se ejecuta 1 vez
print(nc) # 0.3749675669996577
nc = timeit.timeit('fibo(32)', globals = globals(), number = 10)
# se ejecuta 10 veces
print(nc) # 3.6652043369999774 (la anterior x 10)
c = timeit.timeit('mem_fibo(32)', globals = globals(), number = 1)
# con caché 1 vez
print(c) # 0.3642976340001951 (similar a sin saché)
c = timeit.timeit('mem_fibo(32)', globals = globals(), number = 10)
# con cache 10 veces
print(c) # 1.5429995983140543e-06 (rapidísimo)
c = timeit.timeit('mem_fibo(32)', globals = globals(), number = 1000)
# con caché 1000 veces
print(c) # 6.248899990168866e-05 (rapidísimo)
```

Realiza la actividad memoización.py

men_fibo(32) provoca la llamada de memoize(fibo(32)) que cachea fibo(32) en el diccionario (si no está) y retorna dicho resultado. En el diccionario interno de memoize solo se pueden guardar objetos hasheables (método mágico __hash__)

DECORADORES

Los decoradores son funciones de orden superior: aceptan otras funciones como parámetros.

Los decoradores definen lógica interna para satisfacer una necesidad específica de la función que se pasa como parámetro. Esta lógica la realizan mediante una función interna. La función interna accede a los parámetros de la función que se pasa como parámetro mediante args y kwargs.

Por ejemplo: vamos a crear una función decoradora temporiza que indica el tiempo que cuesta ejecutar una función que se le pasa como parámetro.

```
import time
import string
import functools
```

FUNCIÓN DECORADORA

```
def temporiza(func):
    def func_interna(*args, **kwargs):
        inicio = time.time()
        resultado = func(*args, **kwargs)
        tiempo = time.time() - inicio
        print(f'Función {func.__name__} tardó {tiempo}s usando {args} y {kwargs}')
        return resultado
    return func_interna
```

```
def convierte_formato(cad):
    lista_elems = list(cad)
    lista_final = []
    for c in lista_elems:
        elem = c.lower() if c.lower() > 'p' else c.upper()
        lista_final.append(elem)
    return ''.join(lista_final)
```

```
# FUNCIÓN DECORADA
tempo_formato = temporiza(convierte_formato)
```

Realiza la actividad decoradores.py

```
print(tempo_formato(string.ascii_letters))
# Función convierte_formato tardó 1.1444091796875e-05s usando
#('abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ',) y {}
# ABCDEFGHIJKLMNOPqrstuvwxyzABCDEFGHIJKLMNOPqrstuvwxyz

print(tempo_formato("En un lugar de la Mancha"))
# Función convierte_formato tardó 4.291534423828125e-06s usando
# ('En un lugar de la Mancha',) y {}
# EN uN LuGAr DE LA MANCHA
```

DECORADORES

Python facilita el uso de los decoradores de la siguiente forma: el decorador (precedido de @) se añade en la línea superior de la función a decorar y el intérprete lo entiende sin problemas.

```
@temporiza
def suma_elems(elems): #automáticamente decorada
    acc = 0
    for x in elems:
        acc += x
    return acc

@temporiza
@functools.lru_cache(maxsize = 128)
def fibo_recursivo(n):
    if n <= 1:
        return n
    else:
        return fibo_recursivo(n - 1) + fibo_recursivo(n - 2)

print(suma_elems(range(1000)))

print(fibo_recursivo(10))
print(fibo_recursivo(10))
```

Podemos tener más de un decorador por función. La función fibo_recursivo() está decorada con temporiza y una caché de functools.

Realiza la actividad decoradores.py

```
Función suma_elems tardó 2.956390380859375e-05s usando (range(0, 1000),) y {}
499500
Función fibo_recursivo tardó 7.152557373046875e-07s usando (1,) y {}
Función fibo_recursivo tardó 4.76837158203125e-07s usando (0,) y {}
Función fibo_recursivo tardó 1.8358230590820312e-05s usando (2,) y {}
Función fibo_recursivo tardó 2.384185791015625e-07s usando (1,) y {}
...
Función fibo_recursivo tardó 4.76837158203125e-07s usando (8,) y {}
Función fibo_recursivo tardó 0.00011801719665527344s usando (10,) y {}
55

Función fibo_recursivo tardó 4.76837158203125e-07s usando (10,) y {}
```

DOCUMENTACIÓN DE FUNCIONES

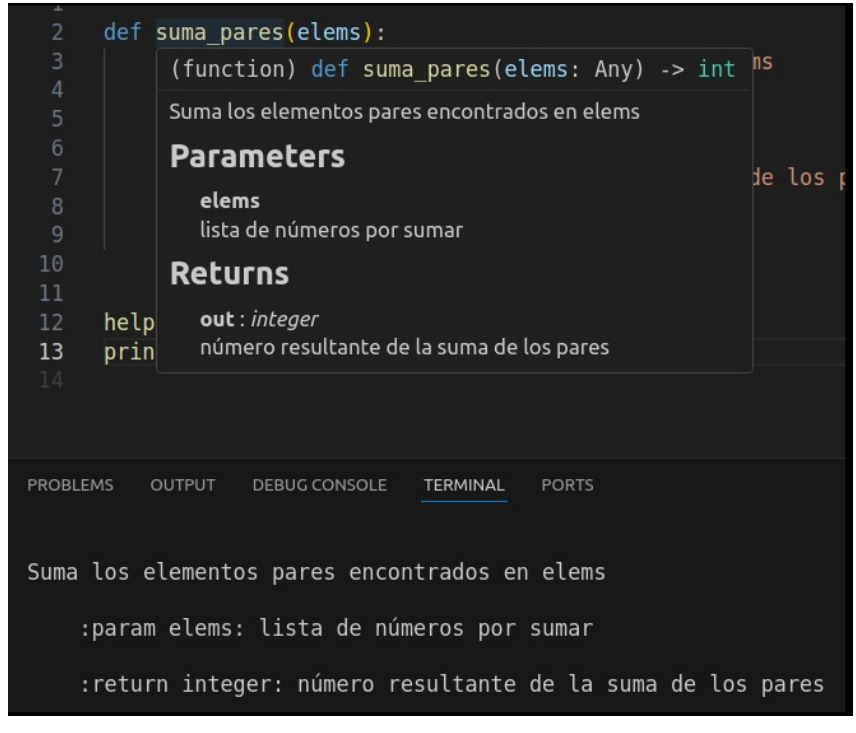
Existe un sistema para documentar las funciones (y clases). Se añade una cadena de caracteres de la definición. Esta cadena suele ser de una sola línea o de múltiples líneas y explica la idea general de la lógica de la función y, opcionalmente, cada uno de los parámetros y el objeto que esta devuelve.
Podemos ver la documentación de una función con help() o con el método mágico __doc__

```
def suma_pares(elems):
    """Suma los elementos pares encontrados en elems

    :param elems: lista de números por sumar

    :returns: integer número resultante de la suma de los pares
    """
    return sum(x for x in elems if not x % 2)

help(suma_pares)
print(suma_pares.__doc__)
```



Realiza la actividad documentacion_funciones.py

FUNCIONES GENERADORAS

Python soporta el uso de objetos generadores capaces de generar secuencias de datos potencialmente muy grandes con un uso de memoria mínimo.

Las funciones generadoras son capaces de crear funciones que generen generadores personalizados.

Una función generadora utiliza yield para devolver el valor de la función (en vez de return).

```
def range_personal(inicio, fin, paso=1):
    final = inicio
    while final < fin:
        yield final
        final += paso

def fibo_generador():
    previo = 0
    actual = 1
    yield 0
    while True:
        yield actual
        previo, actual = actual, actual + previo

print(list(range_personal(0, 20, 3))) # [0, 3, 6, 9, 12, 15, 18]
fibo_g = fibo_generador()
print([next (fibo_g) for _ in range(15)]) # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]

f = range_personal(0, 10, 6)
print(next(f)) # 0
print(next(f)) # 6
print(next(f)) # exhausto: excepción StopIteration
```

Este tipo de funciones son como las corrutinas. La primera vez que se llaman se ejecutan hasta que ejecutan el yield (inclusive) y se paran. La siguiente vez que se llaman continúan donde lo dejaron hasta el siguiente

Puesto que este tipo de funciones son generadores se pueden obtener los elementos siguientes con la función next().

Una vez que el generador se ha quedado exhausto, este no contendrá más elementos y elevará la excepción StopIteration.

Realiza la actividad funciones_generadoras.py